

YUGA LABS · SECURITY REVIEW

Yuga Labs TORCH Review.

A security review of TORCH Review.

CLIENT	PROJECT	AUDITORS	AUDIT WINDOW	FINAL REPORT
Yuga Labs	TORCH Review	Guardian	January 16 – January 27, 2026	February 3, 2026

§ 00

Contents.

40 pages — overview, confidence ranking, scope, 26 findings, disclaimer, and about Guardian.

§ 01	Overview	P. 03
§ 02	Confidence Ranking	P. 04
§ 03	Scope & Classifications	P. 05
	Files in Scope	P. 06
§ 04	Findings	P. 07
	Main Review · 21 findings	P. 09
	Remediation Review · 5 findings	P. 34
§ 05	Disclaimer	P. 39
§ 06	About Guardian	P. 40

§ 01

Overview.

Guardian reviewed TORCH Review across 2 review rounds. The final report includes 2 Critical, 2 High, 1 Medium, 6 Low, 15 Informational findings.

AUDIT FIRM	REPOSITORY
Guardian	yuga-labs/TORCH
CLIENT	METHODOLOGY
Yuga Labs	Smart contract · Manual · Main Review · Remediation Review
PROJECT	MAIN REVIEW COMMIT
TORCH Review	612c8ae2fcc3
PREPARED BY	
Guardian	
AUDIT WINDOW	
January 16 – January 27, 2026	
FINAL REPORT	
February 3, 2026	

1.1 Confidence Ranking

3 / 5

Moderate Confidence

Guardian assigns a Confidence Ranking of 3 out of 5, combining finding severity, remediation outcome, and the reviewer’s assessment of latent risk. The full scale is on the next page.

1	2	3	4	5
VERY LOW CONFIDENCE	LOW CONFIDENCE	MODERATE CONFIDENCE	HIGH CONFIDENCE	VERY HIGH CONFIDENCE

1.2 Findings at a Glance

SEVERITY	TOTAL	PENDING	ACK	PARTIAL	RESOLVED
■ Critical	2	0	0	0	2
■ High	2	0	0	0	2
■ Medium	1	0	1	0	0
■ Low	6	0	0	0	6
■ Informational	15	0	9	0	6
All severities	26	0	10	0	16

§ 02

Confidence Ranking.

Guardian assigns a project-level confidence score using the scale below. The ranking combines finding severity, remediation outcome, and the reviewer's assessment of latent risk.

CONFIDENCE RANKING	DEFINITION AND RECOMMENDATION	RISK PROFILE
5 Very High Confidence	Codebase is mature, clean, and secure. No High or Critical vulnerabilities were found. Follows modern best practices with high test coverage and thoughtful design. Recommendation: code is highly secure at time of audit.	0 High/Critical findings and few Low/Medium severity findings.
4 High Confidence	Code is clean, well-structured, and adheres to best practices. Design patterns are sound and test coverage is strong. Recommendation: suitable for deployment after remediations; consider periodic review with changes.	0-1 High/Critical findings per engagement week and little to no Medium severity findings.
3 Moderate Confidence ASSIGNED	Code is functional but shows gaps in defensive programming or test coverage. Recommendation: remediate the findings and re-review the changes before deployment.	Some High/Critical findings, with various Medium severity findings.
2 Low Confidence	Recurring issues suggest systemic gaps rather than isolated mistakes. Recommendation: remediate, then commission a follow-up review at a frozen commit.	Over 6 High and Critical findings across the engagement.
1 Very Low Confidence	The volume and severity of findings indicate the codebase is not ready for production. Recommendation: substantial rework, then a full re-review.	Over 12 High and Critical findings across the engagement.

§ 03

Scope & Classifications.

Guardian reviewed TORCH Review in `yuga-labs/T0RCH`. The scope includes 338 source lines across 3 files.

3.1 Severity Matrix

	IMP · HIGH	IMP · MED	IMP · LOW
LIK · HIGH	Critical	High	Medium
LIK · MED	High	Medium	Low
LIK · LOW	Medium	Low	Low

IMPACT · HIGH
Significant loss of assets, harm to a group of users, or core protocol functionality is disrupted.

IMPACT · MEDIUM
Small amount of funds can be lost, or ancillary functionality is affected. Users may experience reduced or delayed receipt.

IMPACT · LOW
Unexpected behavior in some functionality that is notable but does not meet the criteria for higher severity.

LIKELIHOOD · HIGH
Possible with reasonable assumptions that mimic on-chain conditions; cost is low compared to amount gained.

LIKELIHOOD · MEDIUM
Possible only in uncommon cases or requires a large amount of capital relative to disruption caused.

LIKELIHOOD · LOW
Unlikely to ever occur in production.

Files in Scope.

FILE	NSLOC	TOTAL
src/BoredApeYachtClub.sol	180	345
src/TheSplit.sol	117	260
src/UserOwnedRoyaltySplitter.sol	41	56

§ 04

Findings.

Grouped by round. Visible IDs reset by severity within each round, so a finding is named the same way in the report as it was in the channel it was written in.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
MAIN REVIEW					
C-01	Royalty Owner Can Steal Royalties Via Reentrancy	Reentrancy	CRITICAL	RESOLVED	p. 9
C-02	Splits Can Be Stolen	Access Control	CRITICAL	RESOLVED	p. 11
H-01	Received Royalties Can Be Artificially Inflated	Validation	HIGH	RESOLVED	p. 12
H-02	Royalties From The Pool Are Counted Twice	Logical Error	HIGH	RESOLVED	p. 14
L-01	Minting Fails If Owner Is Renounced	DoS	LOW	RESOLVED	p. 16
L-02	Royalty Splitter Cannot Update Recipients	Best Practices	LOW	RESOLVED	p. 18
L-03	Updating The Split Leads To Incorrect Accounting	Logical Error	LOW	RESOLVED	p. 19
L-04	Unrestricted External Call In <code>withdrawBlurPool</code>	Best Practices	LOW	RESOLVED	p. 20
L-05	Split Not Working With ERC20s	Unexpected Behavior	LOW	RESOLVED	p. 21
I-01	<code>totalSupply()</code> Can Be Artificially Increased	Best Practices	INFORMATIONAL	ACKNOWLEDGED	p. 22
I-02	Uneven Royalty Distribution Due To <code>TheSplit</code>	Informational	INFORMATIONAL	ACKNOWLEDGED	p. 23
I-03	<code>TransferValidator</code> Config And Assumptions	Configuration	INFORMATIONAL	ACKNOWLEDGED	p. 24
I-04	ERC721C: <code>SupportsInterface</code>	Informational	INFORMATIONAL	RESOLVED	p. 25

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
I-05	ERC721C: EIP Implementation	Informational	INFORMATIONAL	ACKNOWLEDGED	p. 26
I-06	BAYC: BurnAllowed Trust Assumption	Gaming	INFORMATIONAL	ACKNOWLEDGED	p. 27
I-07	BAYC_LEGACY Assignment	Best Practices	INFORMATIONAL	RESOLVED	p. 28
I-08	ERC721C: Validate Transfers	Configuration	INFORMATIONAL	ACKNOWLEDGED	p. 29
I-09	Minting Can Be Optimized	Gas Optimization	INFORMATIONAL	RESOLVED	p. 30
I-10	hasRole() May Be Confusing	Best Practices	INFORMATIONAL	RESOLVED	p. 31
I-11	Gas: Mints Optimisation	Gas Optimization	INFORMATIONAL	ACKNOWLEDGED	p. 32
I-12	Theft Of Royalties Earned Through The Blur Pool	Informational	INFORMATIONAL	ACKNOWLEDGED	p. 33

REMEDIATION REVIEW

M-01	Non-ETH Royalties Distributed Unfairly	Gaming	MEDIUM	ACKNOWLEDGED	p. 34
L-01	ERC721 Rescue In UserOwnedRoyaltySplitter	Logical Error	LOW	RESOLVED	p. 35
I-01	Rescinded Ownership And Split Rescue Functions	Configuration	INFORMATIONAL	ACKNOWLEDGED	p. 36
I-02	Redundant Check In hasRole()	Superfluous Code	INFORMATIONAL	RESOLVED	p. 37
I-03	Old Recipients May Receive Splits	Unexpected Behavior	INFORMATIONAL	RESOLVED	p. 38

CRITICAL **C-01**

RESOLVED

Royalty Owner Can Steal Royalties Via Reentrancy

CATEGORY Reentrancy
LOCATION —
REVIEW Main Review

DESCRIPTION

`UserOwnedRoyaltySplitter` is a contract deployed for each original minter of the Club NFT. Three royalty recipients are configured during deployment.

```
function initialize(address recipient1, address recipient2, address recipient3) external {  
    ...  
    recipientOne = recipient1;  
    recipientTwo = recipient2;  
    recipientThree = recipient3;  
}
```

These recipients are respectively the owner of the original NFT, the royalty machine and the owner of the club NFT contract.

```
UserOwnedRoyaltySplitter(splitter).initialize(msg.sender, royaltyMachine, owner());
```

`UserOwnedRoyaltySplitter.withdraw()` is a permissionless function that sends a third of the available native token balance of the contract to each of the three recipients. The last recipient, the NFT contract owner, is forwarded the whole balance. This is done in case one or more of the previous recipients couldn't receive their funds - then the owner gets everything.

```
function withdraw() external {  
    uint256 oneThird = address(this).balance / 3;  
    (bool success,) = recipientOne.call{value: oneThird}("");  
    (success,) = recipientTwo.call{value: oneThird}("");  
    // remainder gets sent to recipient 3. If either previous recipients cannot receive ether,  
    then recipient 3 will receive an extra allotment.  
    (success,) = recipientThree.call{value: address(this).balance}("");  
    if (!success) revert TransferFailed();  
}
```

C-01 Royalty Owner Can Steal Royalties Via Reentrancy

CONTINUED

However, this mechanism introduces an exploitable reentrancy vector through which royalties can be stolen by the first recipient.

The first recipient can call `withdraw()` and reenter as much time as needed in order to withdraw most, if not all, of the contract balance.

Then all of the calls to the second recipient from the upper call contexts will revert, but this doesn't matter since the `success` flag is ignored.

Only the `success` flag of the call to the third recipient is checked, but this call will always be successful due to it using the whole balance of the contract.

Looking back at `BoredApeYachtClub._deployRoyaltySplitter()`, one can see that the first recipient is the owner of the original NFT.

```
UserOwnedRoyaltySplitter(splitter).initialize(msg.sender, royaltyMachine, owner());
```

The owner of the original NFT can exploit this vector and steal the royalties of the machine and the Club owner.

RECOMMENDATION

Add a `nonReentrant` modifier to `UserOwnedRoyaltySplitter()`

PROOF OF CONCEPT https://github.com/GuardianOrg/TORCH-team1-1768511290822/blob/steal_royalties_reentrancy_POC/test/StealRoyaltiesReentrancy_POC.t.sol

RESOLUTION

Status: Resolved.

CRITICAL **C-02**

RESOLVED

Splits Can Be Stolen

CATEGORY Access Control
LOCATION TheSplit.sol#L115-143
REVIEW Main Review

DESCRIPTION

`TheSplit.claimYugaLabs()` and `TheSplit.claimMachine()` are two permissionless functions that accept an arbitrary `to` parameter and forward the Yuga and machine splits to that address. An attacker can call these functions with their own address and in result steal accumulated splits that should have went to Yuga and the machine.

RECOMMENDATION

Either make the functions callable only by Yuga and the machine or don't allow passing arbitrary `to` and instead hardcode Yuga and the machine.

PROOF OF CONCEPT https://github.com/GuardianOrg/TORCH-team1-1768511290822/blob/steal_splits_POC/test/StealSplits_POC.t.sol

RESOLUTION

Status: Resolved.

HIGH **H-01**

RESOLVED

Received Royalties Can Be Artificially Inflated

CATEGORY Validation
LOCATION TheSplit.sol#L178
REVIEW Main Review

DESCRIPTION

The `totalRoyaltiesReceived` variable in `TheSplit` contract tracks the amount of native token received. It's later used to determine the amount of royalties to be distributed.

```
uint256 newRoyalties = totalRoyaltiesReceived - totalRoyaltiesProcessed;
```

The variable is increased in `receive()` and `claimBlurPool()`. However, `claimBlurPool()` accepts an arbitrary pool and increases the variable with the value returned from its `balanceOf()` function.

```
function claimBlurPool(address pool) external {  
    uint256 claimable = IBlurPool(pool).balanceOf(address(this));  
    if (claimable == 0) revert NothingToClaim();  
    IBlurPool(pool).withdraw(claimable);  
    unchecked {  
        totalRoyaltiesReceived += claimable;  
    }  
    emit RoyaltiesReceived(claimable);  
}
```

A fake contract can be used to return any value. This allows anyone to inflate the received royalties and cause incorrect distribution, stealing from other recipients.

The variable can also be set to its maximum value of `uint256.max` to completely DOS any new royalties.

RECOMMENDATION

Use a hardcoded address of the pool.

PROOF OF CONCEPT https://github.com/GuardianOrg/TORCH-team1-1768511290822/blob/inflate_royalties_POC/test/InflateRoyaltiesPOC.t.sol

H-01 Received Royalties Can Be Artificially Inflated

CONTINUED

— RESOLUTION

Status: Resolved.

HIGH **H-02**

RESOLVED

Royalties From The Pool Are Counted Twice

CATEGORY Logical Error
LOCATION TheSplit.sol#L182-185
REVIEW Main Review

DESCRIPTION

`TheSplit.claimPool()` can be invoked to claim the balance of the contract held in the blur pool.

The `claimable` variable is assigned the value returned from `balanceOf()` and is afterwards received by calling `withdraw()`. Then the `totalRoyaltiesReceived` is increased with that value and will be later used to calculate how much new royalties are to be distributed. After that the `RoyaltiesReceived` even is emitted.

```
function claimBlurPool(address pool) external {
    uint256 claimable = IBlurPool(pool).balanceOf(address(this));
    if (claimable == 0) revert NothingToClaim();
    IBlurPool(pool).withdraw(claimable);
    unchecked {
        totalRoyaltiesReceived += claimable;
    }
    emit RoyaltiesReceived(claimable);
}
```

`BlurPool.withdraw()` sends native tokens to `TheSplit`

```
function withdraw(uint256 amount) external {
    uint256 balance = _balances[msg.sender];
    require(balance >= amount, "Insufficient funds");
    unchecked {
        _balances[msg.sender] = balance - amount;
    }
    (bool success,) = payable(msg.sender).call{value: amount}("");
    require(success, "Transfer failed");
    emit Transfer(msg.sender, address(0), amount);
}
```

H-02 Royalties From The Pool Are Counted Twice

CONTINUED

Because of this, `TheSplit.receive()` will be executed. Inside of it, the `totalRoyaltiesReceived` is increased again and `RoyaltiesReceived` - emitted as well.

```
receive() external payable {  
    totalRoyaltiesReceived += msg.value;  
    emit RoyaltiesReceived(msg.value);  
}
```

Therefore, any royalties claimed by the pool will be counted twice in `totalRoyaltiesReceived` and the event will be emitted twice. This will lead to incorrect royalties distribution for the different receivers.

— RECOMMENDATION

Remove the duplicated logic from `claimBlurPool`

```
function claimBlurPool(address pool) external {  
    uint256 claimable = IBlurPool(pool).balanceOf(address(this));  
    if (claimable == 0) revert NothingToClaim();  
    IBlurPool(pool).withdraw(claimable);  
    - unchecked {  
    - totalRoyaltiesReceived += claimable;  
    - }  
    - emit RoyaltiesReceived(claimable);  
}
```

PROOF OF CONCEPT https://github.com/GuardianOrg/TORCH-team1-1768511290822/blob/double_royalties_POC/test/RoyaltiesDoubleAccountingPOC.t.sol

— RESOLUTION

Status: Resolved.

LOW **L-01**

RESOLVED

Minting Fails If Owner Is Renounced

CATEGORY DoS
LOCATION BoredApeYachtClub.sol#L397
REVIEW Main Review

DESCRIPTION

For the first mint of a `tokenId` the `BoredApeYachtClub` deploys a splitter for the user and passes `owner()` as one of the recipients.

```
function _deployRoyaltySplitter(uint256 tokenId) internal {  
    address payable splitter = payable(address(new UserOwnedRoyaltySplitter{salt:  
bytes32(tokenId)}()));  
    UserOwnedRoyaltySplitter(splitter).initialize(msg.sender, royaltyMachine, owner());  
    emit RoyaltySplitterDeployed(tokenId, msg.sender, address(splitter));  
}
```

Then `UserOwnedRoyaltySplitter.initialize()` reverts if any of the recipients is `address(0)`.

```
function initialize(address recipient1, address recipient2, address recipient3) external {  
    if (recipientOne != address(0) || recipientTwo != address(0) || recipientThree != address(0))  
    {  
        revert AlreadyInitialized();  
    }  
    if (recipient1 == address(0) || recipient2 == address(0) || recipient3 == address(0)) {  
        revert InvalidRecipient();  
    }  
    ...  
}
```

Therefore, once the owner of the contract uses `renounceOwnership()` all new mints will be disabled, as they will start failing with `InvalidRecipient()`.

RECOMMENDATION

If renouncing ownership is not a desirable feature, override the function to revert in order to disable renouncing.

L-01 Minting Fails If Owner Is Renounced

CONTINUED

Otherwise, use another address as fee collector instead of `owner()`.

PROOF OF CONCEPT https://github.com/GuardianOrg/TORCH-team1-1768511290822/blob/renounceDOS_POC/test/RenounceDOS_POC.t.sol

— RESOLUTION

Status: Resolved.

LOW **L-02**

RESOLVED

Royalty Splitter Cannot Update Recipients

CATEGORY Best Practices
LOCATION UserOwnedRoyaltySplitter.sol#L34-35
REVIEW Main Review

DESCRIPTION

Two of the recipients in the `UserOwnedRoyaltySplitter` contract are the `royaltyMachine` and the owner of the club collection. Once set, these recipients cannot be changed, so if a modification happens in the `BAYC` contract, the royalties will keep going to the old addresses.

RECOMMENDATION

Instead of setting `recipientTwo` and `recipientThree` during deployment, read them from `BAYC` any time they are needed.

RESOLUTION

Status: Resolved.

LOW **L-03**

RESOLVED

Updating The Split Leads To Incorrect Accounting

CATEGORY Logical Error
LOCATION TheSplit.sol
REVIEW Main Review

DESCRIPTION

`BoredApeYachtClub.setSplitContract()` can be used by accounts with the `SPLIT_MANAGEMENT_ROLE` role to enable, disable or change the split contract. However, changing this variable leads to incorrect accounting for the old split.

Once changed, `onMint()` won't be called on the old split. This creates a discrepancy between the data in `TheSplit` because the contract reads the storage variables of the `BAYC`. Example:

- There is 400 minter reward to be split across 2 users, each owning 1 token
- The split is changed to another contract
- One of the users mints another NFT, `onMint()` is called only on the new splitter
- When this user calls `TheSplit.claimMinter()` on the old contract, they would be able to claim all of the 400 tokens, because to the split it appears like they own 2 NFT and their accumulated reward hasn't been updated

RECOMMENDATION

Either:

- make `setSplitContract()` callable only once and make `TheSplit` read `machine` and `yugaLabs` from `BAYC` instead of having them as immutables
- introduce a way to freeze the old splitter by caching all the needed variables from BAYC before migrating to a new splitter

RESOLUTION

Status: Resolved.

LOW **L-04**

RESOLVED

Unrestricted External Call In `withdrawBlurPool`

CATEGORY	Best Practices
LOCATION	UserOwnedRoyaltySplitter: 59
REVIEW	Main Review

DESCRIPTION

In `UserOwnedRoyaltySplitter`, the value for `_BLUR_POOL` is assigned at the top but is never used, since `withdrawBlurPool` accepts a user-supplied pool address.

Unlike the split contract, the impact here is limited.

However, allowing the contract to make an external call to an arbitrary user-provided address is still not considered best practice.

RECOMMENDATION

Consider using hardcoded blur pool address instead.

RESOLUTION

Status: Resolved.

LOW **L-05**

RESOLVED

Split Not Working With ERC20s

CATEGORY Unexpected Behavior
LOCATION TheSplit.sol
REVIEW Main Review

— DESCRIPTION

TheSplit is designed to work with native tokens only. Any ERC20s received as royalties will be stuck there since there is no way to take them out.

— RECOMMENDATION

Consider whether support for ERC20s is needed.

— RESOLUTION

Status: Resolved.

INFORMATIONAL **I-01**

ACKNOWLEDGED

totalSupply() Can Be Artificially Increased

CATEGORY Best Practices
LOCATION BoredApeYachtClub.sol#L365
REVIEW Main Review

DESCRIPTION

`BoredApeYachtClub.totalSupply()` returns the amount of underlying NFTs staying in the contract.

```
function totalSupply() external view returns (uint256) {  
    return IERC721(BAYC_LEGACY).balanceOf(address(this));  
}
```

Anyone can donate their `BAYC_LEGACY` NFT token to increase `totalSupply()` without an actual club token being minted.

This is unusual for `ERC721.totalSupply()` and can be misused by third-party integrators.

RECOMMENDATION

Consider having an internal `totalSupply` tracker.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-02

ACKNOWLEDGED

Uneven Royalty Distribution Due To TheSplit

CATEGORY Informational
LOCATION TheSplit.sol
REVIEW Main Review

DESCRIPTION

All royalties coming from a BAYC NFT sale that go to `TheSplit` are split evenly between the original owners of the tokens. Some tokens are traded at a much higher price than others, but due to the way `TheSplit` works, its rightful owner will receive as much as the other participants.

This also creates a MEV opportunity. For example, if a very expensive NFT is bought, an external party can frontrun the royalty transaction and enter the system before that in order to receive a part of that royalty.

RECOMMENDATION

Keep that design in mind and consider if it's acceptable.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-03**

ACKNOWLEDGED

TransferValidator Config And Assumptions

CATEGORY	Configuration
LOCATION	Global
REVIEW	Main Review

— DESCRIPTION

The owner of the new BAYC would have to set the token type and list it on the transfer validator for transfers to be processed.

setTokenTypeOfCollection and a placeholder.

Limitbreak's payment processor enforces the payment of royalties, but the exchange used by users may not. Regular monitoring would be needed to block such operators if BAYC adopts a blacklist approach. As per current documentation, Opensea, X2Y2 enforce it while Blur doesn't.

— RECOMMENDATION

Beware of these considerations

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-04**

RESOLVED

ERC721C: SupportsInterface

CATEGORY	Informational
LOCATION	https://github.com/limitbreakinc/creator-token-standards/blob/980a63b33591d568b6e04b45f37deba05a55f787/src/erc721c/ERC721C.sol#L39-L40
REVIEW	Main Review

— DESCRIPTION

Technically, you should also add support for the IERC165 interface for this to work.

[https://github.com/OpenZeppelin/openzeppelin-](https://github.com/OpenZeppelin/openzeppelin-contracts/blob/239795bea728c8dca4deb6c66856dd58a6991112/contracts/utils/introspection/ERC165.sol#L23)

[contracts/blob/239795bea728c8dca4deb6c66856dd58a6991112/contracts/utils/introspection/ERC165.sol#L23](https://github.com/OpenZeppelin/openzeppelin-contracts/blob/239795bea728c8dca4deb6c66856dd58a6991112/contracts/utils/introspection/ERC165.sol#L23)

Also according to Limitbreak's implementation of ERC721C, they also return support for their legacy interface.

In practice, we've never seen anyone actually use supportsInterface in the first place, but we are noting the difference so you're aware of it.

[https://github.com/limitbreakinc/creator-token-](https://github.com/limitbreakinc/creator-token-standards/blob/980a63b33591d568b6e04b45f37deba05a55f787/src/erc721c/ERC721C.sol#L39-L40)

[standards/blob/980a63b33591d568b6e04b45f37deba05a55f787/src/erc721c/ERC721C.sol#L39-L40](https://github.com/limitbreakinc/creator-token-standards/blob/980a63b33591d568b6e04b45f37deba05a55f787/src/erc721c/ERC721C.sol#L39-L40)

— RECOMMENDATION

Beware of this consideration and if deemed important consider fixing it.

— RESOLUTION

Status: Resolved.

INFORMATIONAL I-05

ACKNOWLEDGED

ERC721C: EIP Implementation

CATEGORY	Informational
LOCATION	Global
REVIEW	Main Review

— DESCRIPTION

The official ERC721C implementation by Limitbreak also includes a feature called `AutomaticValidatorTransferApproval`, which allows the transfer validator to be automatically approved as an operator.

We're not sure whether this is something desired for BAYC, but we wanted to note it in case you weren't aware.

<https://github.com/limitbreakinc/creator-token-standards/blob/980a63b33591d568b6e04b45f37deba05a55f787/src/erc721c/ERC721C.sol#L23-L24>

— RECOMMENDATION

Beware of this feature and decide if its something that should be enabled for BAYC.

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-06

ACKNOWLEDGED

BAYC: BurnAllowed Trust Assumption

CATEGORY	Gaming
LOCATION	Global
REVIEW	Main Review

DESCRIPTION

The new BAYC has an optional burnAllowed feature. If enabled, it allows users to burn their ERC721C token to receive the legacy BAYC.

However, this burn operation is not compatible with a split contract. Even if someone burns their token, there is no hook like `onBurn` (similar to `onMint`), so that address would continue receiving its share of royalties in perpetuity.

If burn is enabled, someone could even snipe NFT exchange order books and do `buy → mint → burn → sell` in a single transaction, while still retaining royalties forever.

They could also burn, transfer (sell), and have the buyer mint, effectively bypassing transfer validations.

RECOMMENDATION

We raised this to BAYC team, and it was noted that this feature most likely won't be used and is a emergency escape hatch.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-07**

RESOLVED

BAYC_LEGACY Assignment

CATEGORY	Best Practices
LOCATION	https://github.com/yuga-labs/TORCH/blob/a71f8f7b69ef2e049f2997ff0586c5441bc1ab65/src/BoredApeYachtClub.sol#L38
REVIEW	Main Review

DESCRIPTION

BAYC_LEGACY value is unnecessarily assigned on top as immutable and then reassigned in constructor.

RECOMMENDATION

Consider keeping one of the assignments.

RESOLUTION

Status: Resolved.

INFORMATIONAL **I-08**

ACKNOWLEDGED

ERC721C: Validate Transfers

CATEGORY Configuration

LOCATION <https://github.com/yuga-labs/TORCH/blob/a71f8f7b69ef2e049f2997ff0586c5441bc1ab65/src/BoredApeYachtClub.sol#L390>

REVIEW Main Review

— DESCRIPTION

The limitbreak's implementation of ERC721C bypasses the validation if caller is validator itself.

```
function _preValidateTransfer(
    address caller,
    address from,
    address to,
    uint256 tokenId,
    uint256 /*value*/ internal virtual override {
    address validator = getTransferValidator();
    if (validator != address(0)) {
    if (msg.sender == validator) {
    return;
    }
    ITransferValidator(validator).validateTransfer(caller, from, to, tokenId);
    }
    }
```

The BAYC's implementation doesn't.

Hence if BAYC uses the default list provided, it may not allow transfers called by validators itself.

— RECOMMENDATION

Beware of this difference, if transfers made from validator is something BAYC wants to allow, either consider adding validator to the list manually OR add override.

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-09

RESOLVED

Minting Can Be Optimized

CATEGORY Gas Optimization
LOCATION BoredApeYachtClub.sol
REVIEW Main Review

DESCRIPTION

The current royalty architecture deploys **one** `UserOwnedRoyaltySplitter` contract per `tokenId` on first mint. While this works correctly, it introduces significant and unnecessary gas costs during minting. Importantly, the deployed splitter is not token-owned in practice:

- The splitter is initialized with the original minter as the primary recipient.
- If burning is disabled, the original minter relationship is permanent.
- All splitters have identical logic and identical recipient structure (minter / machine / Yuga).
- Different `tokenIds` minted by the same user ultimately route royalties to the same address(es).

As a result, per-token splitters provide no additional economic or functional guarantees beyond having a distinct contract address per `tokenId`. Their only real benefit is address-level isolation and per-token royalty history, which is primarily an indexing and analytics concern, not a correctness or security requirement.

RECOMMENDATION

If the offchain benefits from the current approach are not important to you, consider deploying a royalty minter per user only once and adding a mapping that tracks the original owner of each `tokenId` in order to route `royaltyInfo()` correctly.

RESOLUTION

Status: Resolved.

INFORMATIONAL **I-10**

RESOLVED

hasRole() May Be Confusing

CATEGORY	Best Practices
LOCATION	BoredApeYachtClub.sol#L374-376
REVIEW	Main Review

DESCRIPTION

The `BoredApeYachtClub.hasRole()` function is intended to be used by the `CreatorTokenTransferValidator`. If it's used for any other purposes, its return value will likely not be correct. For example, any role besides the default admin will return false. If third party integrators are not aware of this behavior, they may take wrong decisions based on the returned value.

RECOMMENDATION

Consider documenting that this function shouldn't be used, or even revert if the caller is not the transfer validator.

RESOLUTION

Status: Resolved.

INFORMATIONAL **I-11**

ACKNOWLEDGED

Gas: Mints Optimisation

CATEGORY Gas Optimization
LOCATION Global
REVIEW Main Review

— DESCRIPTION

Yuga deploys a new user-owned splitter for each mint. Instead, they could consider using a deterministic clone model, which keeps a single implementation contract with minimal proxy clones deployed for each mint.

This would lead to a reduction in gas costs for each mint, since much smaller bytecode would be deployed.

— RECOMMENDATION

Consider this approach

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-12

ACKNOWLEDGED

Theft Of Royalties Earned Through The Blur Pool

CATEGORY	Informational
LOCATION	Global
REVIEW	Main Review

— DESCRIPTION

The splitter contract includes a function to claim royalties from the Blur pool.

For standard royalty payments, funds are received directly via the receive function and are instantly accounted for, immediately increasing totalReceived.

However, Blur royalties are handled differently, the royalty amount is credited within the Blur pool, but the funds are not transferred to the splitter contract until claimBlurPool is called. Only at that point does totalReceived increase.

If a user mints after a blur accounting has occurred but before `claimBlurPool` is called, they will still receive a share of those previously earned royalties. This results in incorrect royalty distribution and unfair payouts.

— RECOMMENDATION

Since yuga team mentioned that they dont expect blur pool claims, this could be left as it is. Just beware of this case.

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-01**

ACKNOWLEDGED

Non-ETH Royalties Distributed Unfairly

CATEGORY Gaming
LOCATION TheSplit.sol
REVIEW Remediation Review

— DESCRIPTION

The TheSplit contract uses an accumulator pattern that only tracks ETH-based royalties via `totalRoyaltiesReceived` (incremented in `receive()`). When royalties are paid in ERC20 or ERC721 tokens, the Yuga team intends to rescue them via `rescueERC20/rescueERC721`, swap to ETH, and send the ETH back to the splitter.

However as per above, users who mint after the original ERC20/ERC721 was received but before the ETH swap is deposited will also get the same treatment as if those royalties came in after their mint. This means the timing of when non-ETH royalties are converted and deposited as ETH creates arbitrary reward distribution unrelated to when the original royalty was actually earned.

— RECOMMENDATION

There is no easy solution. Even if the Yuga team acts quickly to swap and deposit, the vector exists during the mint period (until all 10,000 BAYC are minted).

Possible mitigations:

1. Accept this as a known limitation during the mint phase
2. Consider a more complex accounting system that tracks "pending non-ETH royalties" separately
3. Pause minting while significant ERC20/ERC721 royalties are being processed (operationally complex)

— RESOLUTION

Status: Acknowledged.

LOW **L-01**

RESOLVED

ERC721 Rescue In UserOwnedRoyaltySplitter

CATEGORY	Logical Error
LOCATION	UserOwnedRoyaltySplitter.sol
REVIEW	Remediation Review

— DESCRIPTION

The TheSplit contract includes both rescueERC20 and rescueERC721 functions (TheSplit.sol:204-217) to handle royalties received in non-ETH tokens.

However, UserOwnedRoyaltySplitter only implements withdrawERC20 (UserOwnedRoyaltySplitter.sol:60-70) and has no mechanism to handle ERC721 tokens that may be sent to user-owned splitter contracts.

If an ERC721 is sent to a UserOwnedRoyaltySplitter as royalties, it will be permanently stuck.

— RECOMMENDATION

Consider adding rescue for ERC721 here as well.

— RESOLUTION

Status: Resolved.

INFORMATIONAL **I-01**

ACKNOWLEDGED

Rescinded Ownership And Split Rescue Functions

CATEGORY	Configuration
LOCATION	https://github.com/GuardianOrg/TORCH-team1-1768511290822/blob/7fe5ea319ab5a9780f6ca4fbed5369a23fac5bc0/src/TheSplit.sol#L128-L129
REVIEW	Remediation Review

— DESCRIPTION

The BAYC contract allows the owner to rescind ownership by setting owner = address(0). In this case, the code intentionally treats RoyaltyMachine as the effective owner.

However, in the Split contract, this same returned owner (RoyaltyMachine when owner == address(0)) is responsible for rescuing stuck ERC20 and ERC721 tokens. If RoyaltyMachine does not implement methods to call the Split contract's rescue functions, those assets become stuck once ownership is rescinded.

— RECOMMENDATION

Beware of this constraint while developing royalty machine.

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-02

RESOLVED

Redundant Check In hasRole()

CATEGORY	Superfluous Code
LOCATION	BoredApeYachtClub.sol#L416
REVIEW	Remediation Review

DESCRIPTION

The `hasRole()` function in the `BAYC` contract reverts if the role being checked is not `address(0)`. However, it checks the role against `address(0)` again on the next line.

```
function hasRole(bytes32 role, address account) external view returns (bool) {  
    if (role != bytes32(0)) revert UseHasAnyRoleInstead();  
    return (role == bytes32(0) && hasAnyRole(account, TRANSFER_VALIDATOR_MANAGEMENT_ROLE));  
}
```

RECOMMENDATION

Remove the second check.

```
function hasRole(bytes32 role, address account) external view returns (bool) {  
    if (role != bytes32(0)) revert UseHasAnyRoleInstead();  
    - return (role == bytes32(0) && hasAnyRole(account, TRANSFER_VALIDATOR_MANAGEMENT_ROLE));  
    + return hasAnyRole(account, TRANSFER_VALIDATOR_MANAGEMENT_ROLE);  
}
```

RESOLUTION

Status: Resolved.

INFORMATIONAL I-03

RESOLVED

Old Recipients May Receive Splits

CATEGORY Unexpected Behavior
LOCATION TheSplit.sol#L124-129
REVIEW Remediation Review

DESCRIPTION

`TheSplit.updateRecipients()` reads `recipientTwo` and `recipientThree` from `BAYC` and updates its storage variables. In `BAYC`, these are the `royaltyMachine` and the owner, or if there is no owner it's twice the royalty machine.

If the owner of the `BAYC` contract changes one of the recipients before claiming their share, the split royalties may be forwarded to the old addresses before `updateRecipients()` is called.

RECOMMENDATION

Consider removing the `updateRecipients()` function and instead fetch the appropriate recipient on each call to `claimYugaLabs()` and `claimMachine()`.

RESOLUTION

Status: Resolved.

§ 05

Disclaimer.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts Guardian to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. Guardian's position is that each company and individual are responsible for their own due diligence and continuous security. Guardian's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by Guardian is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis.

Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

Notice that smart contracts deployed on the blockchain are not resistant from internal/external exploit. Notice that active smart contract owner privileges constitute an elevated impact to any smart contract's safety and security. Therefore, Guardian does not guarantee the explicit security of the audited smart contract, regardless of the verdict.

§ 06

About Guardian.

Guardian is a smart contract security firm. Every engagement is worked by two independent teams competing on the same scope, each with its own tooling and methods, so no single reviewer's blind spot survives.

\$45B+DIGITAL ASSETS UNDER
REVIEW**200+**

ENGAGEMENTS COMPLETED

500+CRITICAL & HIGH FINDINGS
REPORTED

Reach us at audits@guardianaudits.com.